OptiCrop: Smart Agricultural Production Optimization Engine Using Machine Learning

Project Description

The project Smart Agricultural Production Optimization Engine" aims to develop an advanced software system that utilizes data-driven insights to optimize agricultural production for different crops. By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil temperature, humidity, pH, rainfall, and crop types, OptiCrop seeks to provide intelligent recommendations to farmers for maximizing yields and resource efficiency. The knowledge and insights gained through the OptiCrop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance the understanding of crop-environment interactions and inform the development of evidence-based agricultural strategies. The OptiCrop project revolutionizes agricultural practices by providing farmers with a smart production optimization engine. By integrating key environmental factors and crop-specific recommendations, OptiCrop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustainable agricultural practices

Scenarios

Scenario 1: Smart Crop Recommendation for Farmers

A farmer enters soil and environmental details such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

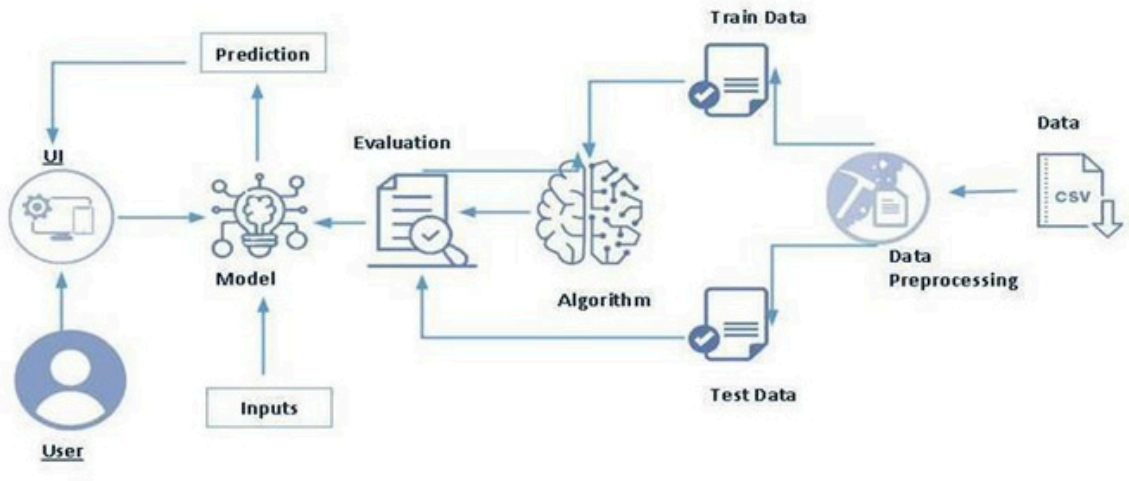
Scenario 2: Crop Suitability and Environmental Assessment

A user wants to understand whether the current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential.

Scenario 3: Agricultural Research and Policy Planning

An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production. The platform helps in making data-driven decisions for sustainable farming strategies and resource optimization.

Technical Architechture



OptiCrop is a machine learning-based agricultural recommendation system designed to optimize crop production using environmental and soil parameters. The project integrates machine learning algorithms

with a Flask web application to provide intelligent crop recommendations based some factors. The project aims to assist farmers, agricultural researchers, agribusiness companies, and policymakers in making data-driven farming decisions that improve productivity, sustainability, and resource utilization.

Instuctions given:

1. Environment Setup & Dependency Configuration

Set up the OptiCrop project using Python 3.10+ and configure the virtual environment. Required libraries include:

Flask

Scikit-learn

Pandas & NumPy

Matplotlib & Seaborn

Pickle

Jupyter Notebook

HTML, CSS, Bootstrap

Git Version Control

2. Machine Learning Model Selection & Architecture

Analyze and preprocess the agricultural dataset containing soil nutrients and environmental parameters.

Implement and compare multiple machine learning algorithms such as:

K-Nearest Neighbors (KNN)

Logistic Regression

Decision Tree

Random Forest

K-Means Clustering

Train and evaluate models using parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. Select the best-performing model and save it in .pkl format for deployment.

3. Core Backend Development

Develop the backend system using Flask and modular Python functions.

Build modules for:

Data preprocessing and cleaning

Model training and evaluation

Crop recommendation prediction

Input validation and preprocessing

The backend processes user input values and generates intelligent crop recommendations using the trained machine learning model.

4. Data Processing & Model Persistence

Implement preprocessing techniques for handling missing values, feature scaling, and categorical transformations.

Store the trained machine learning model using Pickle (.pkl) for reuse during prediction.

Maintain datasets and processed outputs for future agricultural analysis and model improvements.

5. Flask Frontend UI Development

Design an interactive and responsive web interface using HTML, CSS, and Bootstrap.

Create input forms for agricultural parameters such as:

Nitrogen (N)

Phosphorous (P)

Potassium (K)

Temperature

Humidity

pH Level

Rainfall

Integrate prediction buttons to generate crop recommendations instantly.

Display prediction results dynamically on the result page with a clean and user-friendly interface.

6. Testing & Deployment

Perform model evaluation and system testing to ensure accurate crop recommendations.

Validate the Flask application locally using:

Flask Development Server

Browser-based testing

Model accuracy evaluation metrics

Deploy the application locally to provide real-time agricultural recommendations through the web interface.

Outcome:

By completing this project, you will:

Build an AI-powered agricultural recommendation system.

Apply machine learning algorithms for crop prediction and optimization.

Develop a Flask-based web application with an interactive frontend.

Perform data preprocessing, model training, evaluation, and deployment.

Provide intelligent farming recommendations for improving agricultural productivity, sustainability, and resource utilization.

Entity Relationship Diagram

The ER diagram represents the core entities involved in the Loan Approval Prediction System and illustrates how they interact with each other. It provides a clear structure for organizing applicant information, loan requests, prediction results, and machine learning model outputs within the database.

Entities Involved

The ER diagram features seven primary entities:

User

SoilData

Crop

Dataset

MLModel

Prediction

Report

Primary Keys

Each entity is uniquely identified by its primary key:

User: user_id

SoilData: soil_id

Crop: crop_id

Dataset: dataset_id

MLModel: model_id

Prediction: prediction_id

Report: report_id

Relationships

User to SoilData: One user can submit multiple soil data records for crop analysis (1 to Many).

SoilData to Prediction: A single soil data entry can generate one crop prediction result (1 to One).

Crop to Prediction: One crop can be recommended in multiple prediction results (1 to Many).

MLModel to Prediction: One machine learning model can generate multiple prediction records (1 to Many).

Dataset to MLModel: A single dataset can be used to train multiple machine learning models (1 to Many).

Prediction to Report: One prediction can generate multiple agricultural reports and recommendations (1 to Many).

Foreign Keys

SoilData references User via user_id.

Prediction references SoilData via soil_id.

Prediction references Crop via crop_id.

Prediction references MLModel via model.id.

Report references Prediction via predictio_id.

Cardinality

A single user can submit multiple soil data records for agricultural analysis. Each soil data entry is processed by the machine learning model to generate crop prediction results. A crop can appear in multiple predictions, while a single machine learning model can generate predictions for many users and soil conditions. Each

prediction can further generate multiple agricultural reports containing summaries and farming recommendations.

Normalization and Structure

The ER diagram is normalized to separate user information, soil parameters, crop details, datasets, machine learning models, prediction records, and reports into different entities. This structure reduces redundancy, improves scalability, and ensures efficient management of agricultural and prediction-related data.

Use Case Coverage

This ER model supports the core functionalities of the OptiCrop Smart Agricultural Production Optimization System, including:

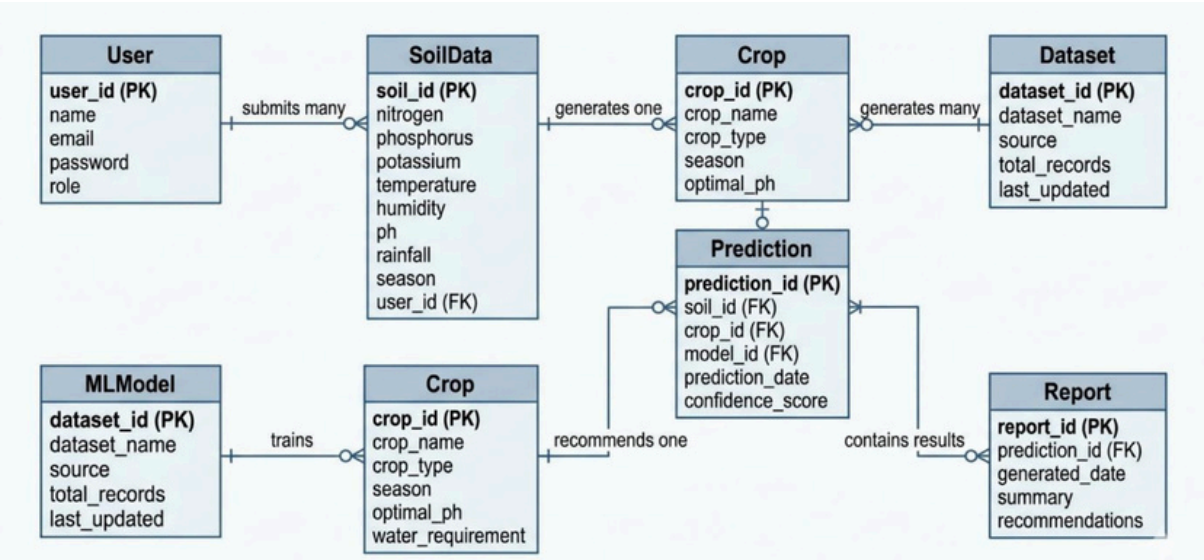
Collecting and managing soil and environmental data from users.

Predicting suitable crops using machine learning algorithms.

Managing datasets and trained machine learning models.

Generating intelligent crop recommendations and prediction reports.

Supporting sustainable farming practices and data-driven agricultural decision-making.



Pre requisites :

The project was developed using a set of powerful Python-based tools and libraries for data processing, machine learning, visualization, and deployment. These technologies provide the foundation for efficient model development, analysis, and application deployment.

Anaconda Navigator: A desktop graphical user interface used to launch applications and manage Python packages and environments for data science. https://www.anaconda.com/download

PyCharm: An Integrated Development Environment (IDE) specifically designed for Python programming, offering code analysis and debugging tools. https://www.jetbrains.com/pycharm/

NumPy: Core library for numerical computing in Python, providing support for large, multi-dimensional arrays and mathematical functions. https://numpy.org/doc/stable/

Pandas: Data manipulation and analysis library offering data structures and operations for manipulating numerical tables and time series. https://pandas.pydata.org/docs/

Scikit-learn: Machine learning library featuring classification, regression, and clustering algorithms like support vector machines and random forests. https://scikit-learn.org/stable/

Matplotlib: Comprehensive library for creating static, animated, and interactive visualizations in Python. https://matplotlib.org/stable/

Seaborn: Data visualization library based on matplotlib that provides a high-level interface for drawing attractive statistical graphics. https://seaborn.pydata.org/

Flask: A lightweight WSGI web application framework used to build web APIs and deploy machine learning models. https://flask.palletsprojects.com/

Project Workflow :

Epic 1: Define Problem and Understanding

Story 1: Identify and define the agricultural problem that the crop recommendation system aims to solve.

Story 2: Gather and analyze business requirements to understand project objectives and expected outcomes.

Story 3: Conduct a literature survey to study existing crop recommendation techniques and machine learning approaches.

Story 4: Analyze the social and business impact of implementing an AI-driven crop recommendation solution.

Epic 2: Data Collection and Analysis

Story 1: Download and collect the agricultural dataset from reliable sources.

Story 2: Import the required Python libraries for data analysis, visualization, and machine learning.

Story 3: Read and explore the dataset to understand its structure, features, and target variables.

Story 4: Perform univariate analysis to examine the distribution and characteristics of individual features.

Story 5: Conduct bivariate analysis to identify relationships between agricultural parameters and crop suitability.

Story 6: Perform multivariate analysis to discover patterns and interactions among multiple variables.

Epic 3: Data Pre-Processing

Story 1: Check for null values and handle missing data to improve dataset quality.

Story 2: Detect and treat outliers that may affect model performance.

Story 3: Extract seasonal crop information and prepare relevant features for analysis.

Story 4: Split the dataset into training and testing sets for model development and evaluation.

Epic 4: Model Building

Story 1: Apply K-Means Clustering to identify patterns and group similar agricultural conditions.

Story 2: Train a Logistic Regression model to predict suitable crops based on environmental parameters.

Story 3: Evaluate model performance using appropriate metrics and compare results.

Story 4: Save the best-performing model for deployment and generate crop predictions based on user-provided inputs.

Epic 5: Application Building

Story 1: Design and develop HTML pages to create an interactive and user-friendly interface.

Story 2: Build the Python backend code and integrate it with the trained machine learning model.

Story 3: Run, test, and validate the application to ensure accurate crop recommendations and smooth functionality.

Epic 1: Define Problem and Understanding

This phase focuses on identifying agricultural challenges, defining project objectives, analyzing crop production requirements, conducting literature reviews, and evaluating the impact of AI-driven crop optimization for sustainable farming decisions.

Specify the Business Problem

The main objective of this phase is to clearly identify and understand the business problem addressed by the system. The project focuses on solving challenges faced in modern agriculture such as poor crop selection, inefficient resource utilization, changing environmental conditions, and reduced farming productivity. Farmers often struggle to determine the most suitable crop based on soil nutrients and climate conditions, which can lead to financial losses and low agricultural yield.

This phase involves analyzing the agricultural domain and identifying the important environmental and soil parameters that directly affect crop growth and productivity. Key factors such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH value, rainfall, and seasonal conditions are studied to understand their impact on crop recommendation and agricultural optimization.

The system aims to provide an intelligent and data-driven solution that assists farmers, agricultural researchers, agribusiness organizations, and policymakers in making better farming decisions. By integrating machine learning techniques and predictive analytics, the platform can recommend the most suitable crops for specific environmental conditions and help improve agricultural efficiency, sustainability, and productivity.

This phase also includes understanding the practical applications, business requirements, user needs, and expected outcomes of the system. The knowledge gathered during this stage forms the foundation for data collection, machine learning model development, application design, and deployment of the agricultural recommendation system.

Business Requirements

The system should accurately analyze agricultural and environmental data to generate intelligent crop recommendations for farmers and agricultural stakeholders. The application must process important soil and climate parameters including Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, rainfall, pH level, and seasonal conditions to identify the most suitable crop for cultivation.

The platform should provide a simple, responsive, and user-friendly web interface where users can easily input agricultural conditions and receive real-time crop prediction results. The application must ensure smooth integration between the machine learning model and the web application to provide fast, reliable, and accurate recommendations.

The system should support multiple machine learning algorithms such as K-Nearest Neighbors (KNN), Logistic Regression, Decision Tree, Random Forest, and K-Means Clustering for training and evaluation. It should also include data preprocessing, feature scaling, model evaluation, and prediction visualization functionalities to improve performance and usability.

Additionally, the platform should help optimize the usage of water, fertilizers, and soil nutrients while reducing crop failure risks caused by poor crop selection and environmental uncertainty. The application should maintain scalability, reliability, and flexibility for future enhancements such as advanced predictive models, larger agricultural datasets, cloud deployment, and smart farming integrations.

Literature Survey

This phase involves conducting a detailed study of existing agricultural recommendation systems, crop prediction technologies, and machine learning-based smart farming solutions. Various research

papers, journals, case studies, online resources, and agricultural technology platforms are analyzed to understand how artificial intelligence and data analytics are currently being used in agriculture.

The literature survey focuses on understanding different machine learning algorithms used for crop recommendation, yield prediction, soil analysis, and environmental monitoring. Existing techniques such as Decision Trees, Random Forest, KNN, Logistic Regression, Neural Networks, and clustering algorithms are studied to compare their performance, advantages, and limitations in agricultural applications.

The survey also examines data preprocessing techniques, feature engineering methods, dataset balancing approaches, and evaluation metrics commonly used in agricultural machine learning systems. Existing smart farming platforms and precision agriculture technologies are analyzed to identify gaps, challenges, and opportunities for improvement.

This phase helps in understanding current technological advancements, identifying research limitations, and selecting suitable methodologies for developing an efficient agricultural production optimization system. The insights gathered from the literature survey form the foundation for model development, system architecture, prediction optimization, and future agricultural innovations.

Social and Business Impact

The project can create a significant positive impact on both the agricultural sector and society by supporting intelligent and data-driven farming practices. By providing accurate crop recommendations based on soil and environmental conditions, the system can help farmers improve agricultural productivity, reduce farming risks, and increase profitability. Better crop selection can lead to higher yields, improved food production, and more efficient utilization of agricultural resources.

The system also promotes sustainable farming practices by helping optimize the usage of water, fertilizers, pesticides, and soil nutrients. This can reduce environmental damage, minimize resource wastage, and support ecological balance. The platform contributes to precision agriculture and smart farming adoption by integrating machine learning technologies into traditional farming processes.

From a business perspective, the system can benefit agribusiness companies, agricultural researchers, policymakers, and farming organizations by providing valuable agricultural insights and predictive decision support. The project can help improve agricultural planning, supply chain management, crop monitoring, and resource allocation strategies.

The platform also has the potential to support rural development and improve the economic condition of farmers by reducing financial losses caused by unsuitable crop selection and environmental uncertainty. In the long term, the project contributes to food security, technological advancement in agriculture, and the adoption of intelligent farming solutions for sustainable agricultural growth.

Epic 2: Data Collection and Analysis

This phase focuses on gathering relevant data from reliable sources and analyzing it to extract meaningful insights. The collected data is processed and prepared to support accurate decision-making and model development.

Download the Dataset

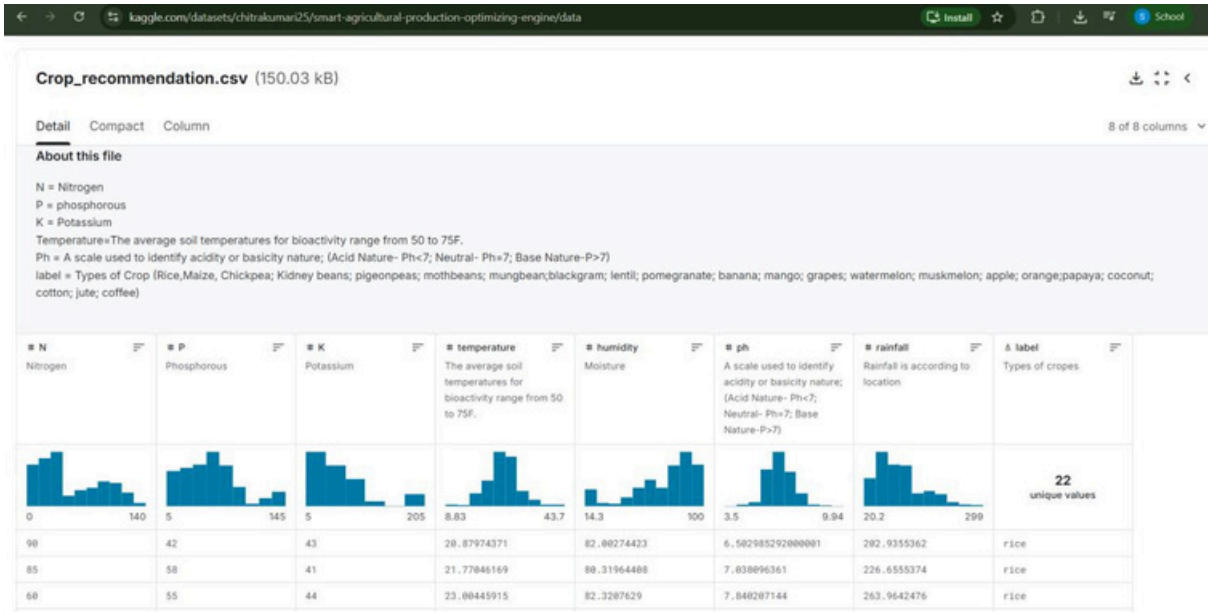
Popular open-source agricultural datasets are available through platforms such as Kaggle and the UCI Machine Learning Repository. For this project, the ***Crop_recommendation.csv*** dataset is used,

which is downloaded from Kaggle for agricultural prediction and crop recommendation analysis. Use the link below to access the dataset:

Dataset Link:

https://www.kaggle.com/datasets/chitrakumari25/smart-agricultural-production-optimizing-engine

Note: There are multiple techniques available for understanding and analyzing agricultural data. This project uses a representative agricultural dataset and selected analytical methods for crop recommendation and smart farming prediction workflows. Additional visualization and analysis techniques may also be implemented as required.



Importing the Libraries

The required Python libraries are imported to perform agricultural data analysis, machine learning operations, and visualization within the OptiCrop system. Libraries such as NumPy and Pandas are used for numerical computation and dataset handling, while Matplotlib and Seaborn are used for graphical visualization and analytical plotting.

Scikit-learn modules are integrated for machine learning model development, clustering, dataset splitting, and performance evaluation. The visualization style used in this project is FiveThirtyEight (fivethirtyeight), which helps generate clean and professional analytical graphs for agricultural data interpretation and crop prediction

```
import pandas as pd
import numpy as np
pd.set_option('max_colwidth', 20)
pd.set_option('display.max_columns',None)
pd.set_option('display.max_rows',50)
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
plt.rcParams['figure.figsize'] = (12,8)
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
from ipywidgets import interact
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Read the dataset:

The agricultural dataset is loaded into the OptiCrop system using the Pandas read_csv() function for further preprocessing and analysis. The dataset contains important agricultural parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, rainfall, and crop labels required for intelligent crop recommendation workflows.

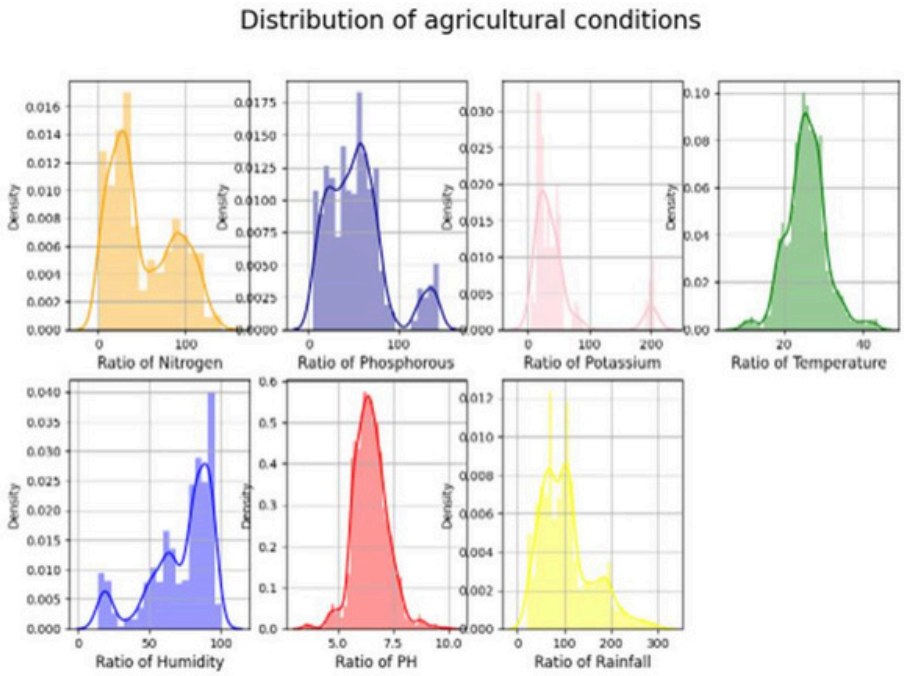
After loading the dataset, the initial records are displayed using the head() function to understand the dataset structure, feature columns, and agricultural data distribution. Proper dataset reading and validation ensure smooth machine learning analysis and accurate crop prediction processing.

```
data=pd.read_csv('/content/gdrive/MyDrive/SmartBridge/Projects/OptiCrop/Crop_recommendation.csv')
data.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

Univariate Analysis

Univariate analysis is used to study individual agricultural features within the crop recommendation dataset. Visualization techniques such as distplot and countplot are used to analyze the distribution of environmental parameters including Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, and rainfall. This analysis helps understand agricultural data patterns required for intelligent crop prediction and smart farming optimization.

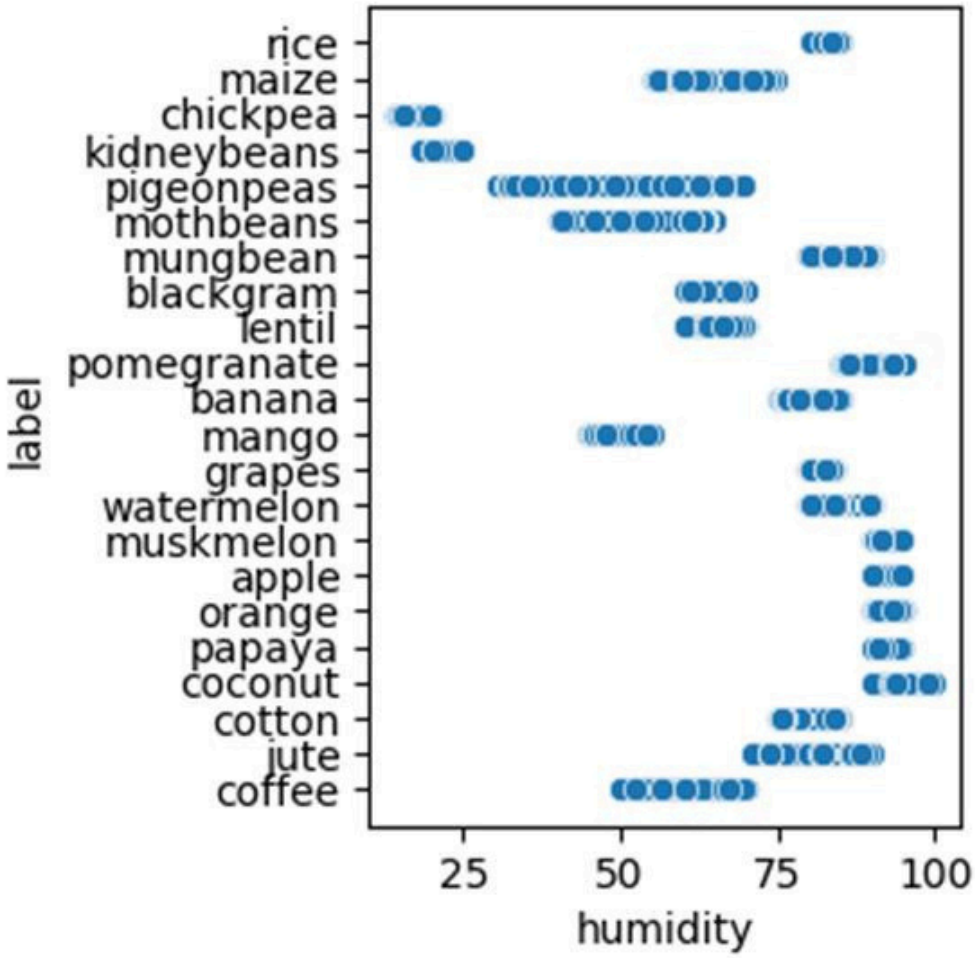


Bivariate Analysis

Bivariate analysis is used to identify relationships between two agricultural features within the crop recommendation dataset. In this analysis, the relationship between humidity and crop labels is visualized using scatter plots to understand environmental impact on crop prediction and agricultural pattern analysis.

```
plt.subplot(2,4,7)
sns.scatterplot(x=df['humidity'],y=df['label'])
```

```
<Axes: ><Axes: xlabel='humidity', ylabel='label'>
```

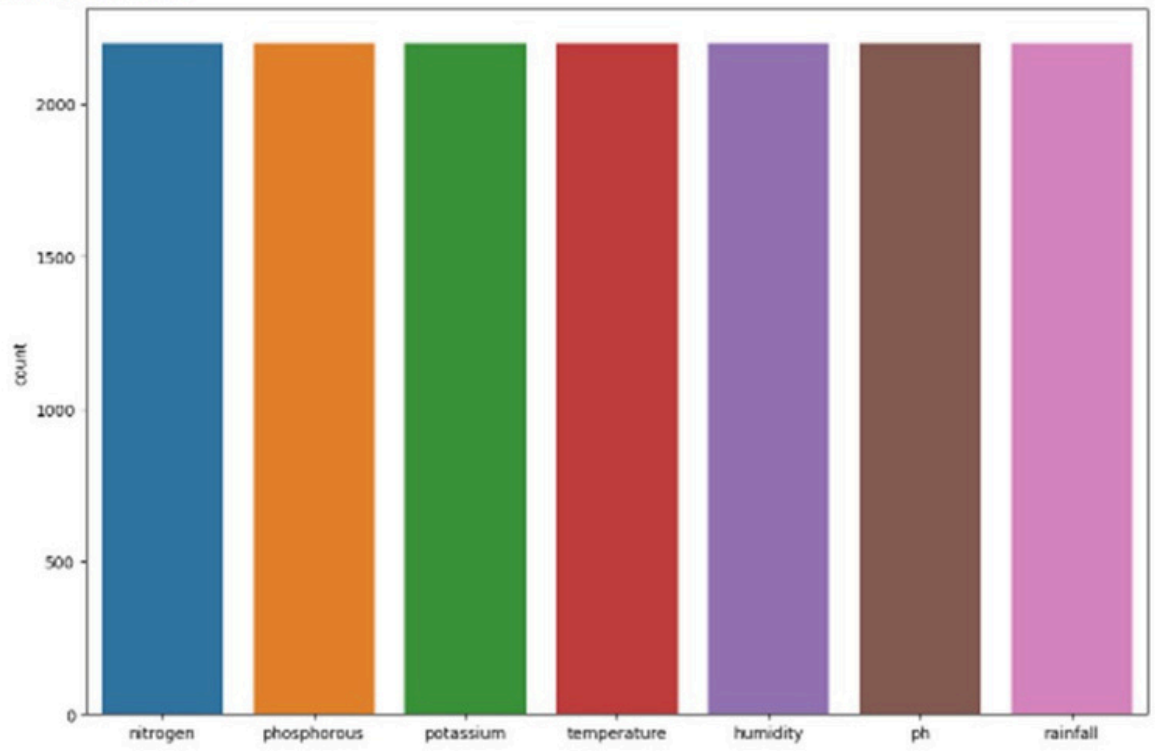


Multivariate Analysis

Multivariate analysis is used to study relationships among multiple agricultural features simultaneously within the crop recommendation dataset. Visualization techniques such as countplot are used to analyze environmental parameters including Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, and rainfall. This analysis helps identify agricultural patterns and supports intelligent crop prediction and smart farming optimization workflows.

```
sns.countplot(df)
```

```
<Axes: ylabel='count'>
```



Descriptive Analysis

```
df.describe()
```

	nitrogen	phosphorous	potassium	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

Epic 3: Data Pre-processing

This phase focuses on preparing the agricultural dataset for model training by handling missing values, treating outliers, extracting relevant features, and splitting the data into training and testing sets to improve prediction accuracy and data quality.

Checking for Null Values

The dataset is examined for missing and null values before preprocessing and machine learning operations. Functions such as df.shape, df.info(), and df.isnull().sum() are used to analyze dataset structure, data types, and null records. This step helps ensure clean and reliable agricultural data for accurate crop prediction and smart farming analysis.

```
[8]  df.isnull().sum()
```

```
nitrogen       0
phosphorous    0
potassium      0
temperature    0
humidity       0
ph             0
rainfall       0
label          0
dtype: int64
```

```
[5]  data.shape
```

```
(2200, 8)
```

```
[4]  data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   N            2200 non-null   int64  
 1   P            2200 non-null   int64  
 2   K            2200 non-null   int64  
 3   temperature  2200 non-null   float64
 4   humidity     2200 non-null   float64
 5   ph           2200 non-null   float64
 6   rainfall     2200 non-null   float64
 7   label        2200 non-null   object 
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB
```

Handling Outliers

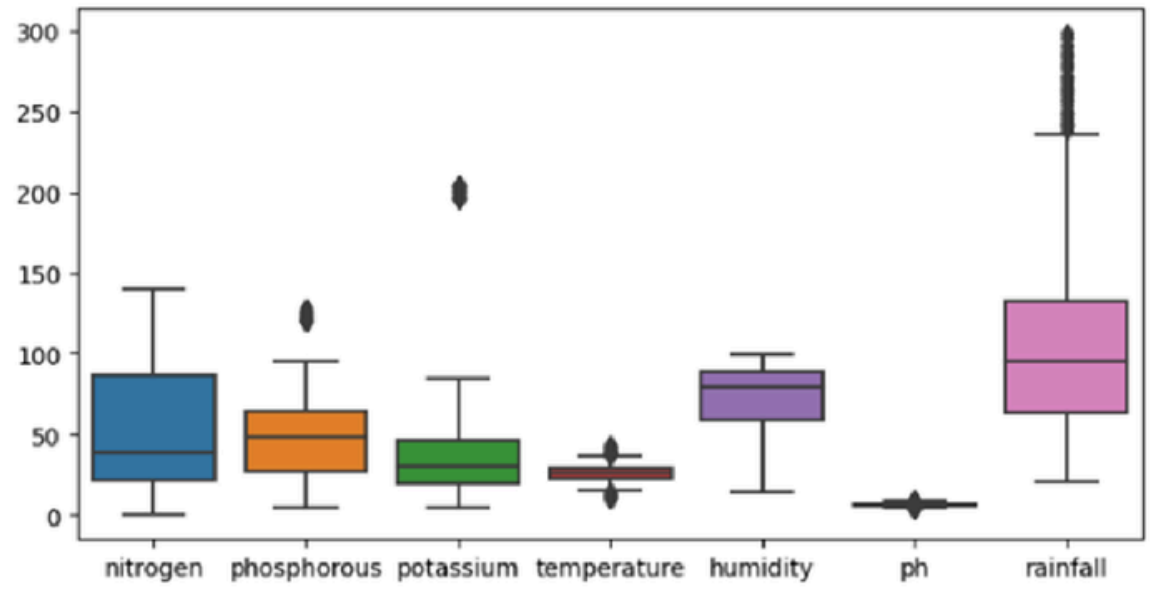
Outliers are identified visually using boxplots. Upper and lower bounds are calculated mathematically using the Interquartile Range (IQR):

- ***Upper Bound*** = Q3 + 1.5 × IQR
- ***Lower Bound*** = Q1 − 1.5 × IQR

A boxplot reveals that the Potassium feature contains outliers. Log transformation is then applied to normalize its distribution.

```
plt.figure(figsize=(8,4))
sns.boxplot(df)
```

```
<Figure size 800x400 with 0 Axes><Axes: >
```




```
Q1 = df['phosphorous'].quantile(0.25)
Q3 = df['phosphorous'].quantile(0.75)
IQR = Q3 - Q1    #IQR is interquartile range.

filter = (df['phosphorous'] >= Q1 - 1.5 * IQR) & (df['phosphorous'] <= Q3 + 1.5 *IQR)
df=df.loc[filter]
```

Extracting Seasonal Crops

Crops are grouped and analyzed based on seasonal environmental conditions such as temperature, humidity, and rainfall. This process helps identify suitable crops for summer, winter, and rainy seasons to improve prediction accuracy and generate season-based agricultural recommendations.

```
print("Summer crops")
print(df[(df['temperature']>30) & (df['humidity']>50)]['label'].unique())
print("-----------------------------------------------")
print("Winter crops")
print(df[(df['temperature']<20) & (df['humidity']>30)]['label'].unique())
print("-----------------------------------------------")
print("Rainy crops")
print(df[(df['rainfall']>200) & (df['humidity']>50)]['label'].unique())
print("-----------------------------------------------")
```

```
Summer crops
['pigeonpeas' 'mothbeans' 'blackgram' 'mango' 'grapes' 'orange' 'papaya']
-----------------------------------------------
Winter crops
['maize' 'pigeonpeas' 'lentil' 'pomegranate' 'grapes' 'orange']
-----------------------------------------------
Rainy crops
['rice' 'papaya' 'coconut']
```

Splitting Data into Train and Test Sets

The dataset is divided into feature variables (X) and target variable (y) for machine learning model training. The train_test_split() function from Scikit-learn is used to split the dataset into training and testing sets with appropriate test size and random state values. This process helps evaluate model performance and improves prediction reliability.

```
y=df['label']
x=df.drop(['label'],axis=1)

print("Shape of x",x.shape)
print("Shape of x=y",y.shape)
```

```
Shape of x (2062, 7)
Shape of x=y (2062,)
```

```
from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test =train_test_split(x,y,test_size=0.2,random_state=0)

print("The shape of x train",x_train.shape)
print("The shape of x test",x_test.shape)
print("The shape of y train",x_train.shape)
print("The shape of y test",x_test.shape)
```

```
The shape of x train (1649, 7)
The shape of x test (413, 7)
The shape of y train (1649, 7)
The shape of y test (413, 7)
```

Epic 4: Model Building

This phase focuses on training machine learning models such as K-Means Clustering and Logistic Regression to analyze crop patterns and predict suitable crops. The models are evaluated using performance metrics, and the best-performing model is saved for deployment.

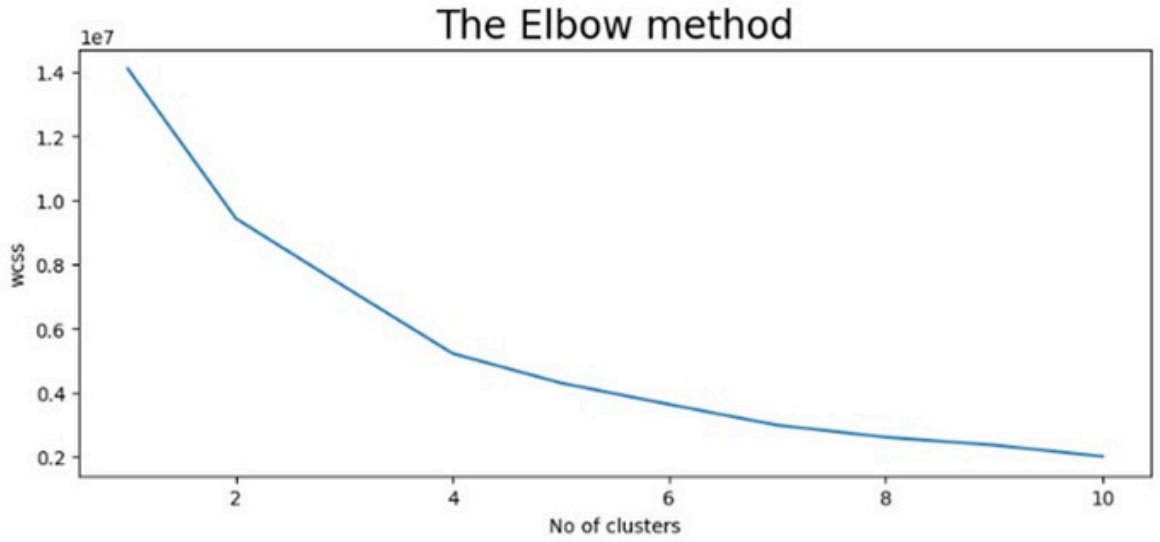
K-Means Clustering

The K-Means clustering algorithm is applied to group similar agricultural patterns based on environmental features. The Elbow Method is used to determine the optimal number of clusters by analyzing WCSS values. The model is trained using the fit() and fit_pridect() methods to identify meaningful crop groupings for intelligent agricultural analysis.

```
#el-bow method used to find out no of clusters and determine the optimum number of clusters within the dataset.
plt.rcParams['figure.figsize']=(10,4)
wcss=[]
for i in range(1,11):
    km=KMeans(n_clusters=i,init="k-means++",max_iter=300,n_init=10,random_state=0)
    km.fit(x)
    wcss.append(km.inertia_)

plt.plot(range(1,11),wcss)
plt.title("The Elbow method",fontsize=20)
plt.xlabel("No of clusters")
plt.ylabel("wcss")
plt.show()
```

Elbow Graph: The Elbow Graph is used to determine the optimal number of clusters by plotting the Within-Cluster Sum of Squares (WCSS) against different cluster values and identifying the point where the rate of decrease sharply changes, known as the "elbow point." It is demonstrated below:



Training the model: The selected machine learning algorithm is trained using the prepared training dataset to learn patterns and make accurate predictions based on input features.

```
km=KMeans(n_clusters=4,init="k-means++",max_iter=300,n_init=10,random_state=0)
y_means=km.fit_predict(x)

a=df['label']
y_means=pd.DataFrame(y_means)
z=pd.concat([y_means,a],axis=1)
z=z.rename(columns={0:'cluster'})


print("lets check the results after applying the K-Means clustering analysis \n")
print("Crops in First cluster:",z[z['cluster']==0]['label'].unique())
print("_____________________________________________________________________")

print("Crops in Second cluster:",z[z['cluster']==1]['label'].unique())

print("_____________________________________________________________________")

print("Crops in Third cluster:",z[z['cluster']==2]['label'].unique())

print("_____________________________________________________________________")

print("Crops in Fourth cluster:",z[z['cluster']==3]['label'].unique())
```

```
lets check the results after applying the K-Means clustering analysis

Crops in First cluster: ['rice' 'pigeonpeas' 'apple' nan 'orange' 'papaya' 'coconut' 'cotton'
 'jute']
_____________________________________________________________________
Crops in Second cluster: ['maize' 'banana' nan 'grapes' 'watermelon' 'muskmelon' 'orange' 'papaya'
 'coconut' 'cotton' 'jute']
_____________________________________________________________________
Crops in Third cluster: ['maize' 'chickpea' 'kidneybeans' 'pigeonpeas' 'mothbeans' 'mungbean'
 'blackgram' 'lentil' 'pomegranate' 'mango' 'muskmelon' 'apple' nan
 'orange' 'papaya']
_____________________________________________________________________
Crops in Fourth cluster: [nan 'grapes' 'muskmelon']
```

Logistic Regression

The Logistic Regression algorithm is implemented for crop prediction using agricultural environmental features. The model is trained using the fit() method and predictions are generated using the predict() function. Model performance is evaluated to analyze prediction accuracy and crop recommendation capability.

```
# LOGUSTIC REGRESSION
from sklearn.linear_model import LogisticRegression

model=LogisticRegression()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
▾ LogisticRegression
LogisticRegression()
```

Evaluating Model Performance and Saving the Best Model

All models are evaluated and compared using important classification metrics such as ***Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC Score***. These metrics help in identifying the model that provides the most accurate and reliable predictions for the dataset.

After evaluating the performance of all trained models, the best-performing model is selected based on its overall classification results and generalization capability. The finalized model is then serialized using ***pickle.dump()*** and stored as ***model.pkl*** so that it can be reused without retraining during deployment in the Flask web application.

Checking model performance:

```
from sklearn.metrics import classification_report
cr=classification_report(y_test,y_pred)
print(cr)
```

	precision	recall	f1-score	support
apple	1.00	1.00	1.00	4
banana	1.00	1.00	1.00	22
blackgram	0.83	0.90	0.86	21
chickpea	1.00	1.00	1.00	24
coconut	1.00	1.00	1.00	19
coffee	1.00	0.95	0.97	20
cotton	0.91	0.91	0.91	22
grapes	1.00	1.00	1.00	6
jute	0.88	0.82	0.85	17
kidneybeans	0.83	1.00	0.91	20
lentil	0.94	0.94	0.94	17
maize	0.83	0.88	0.86	17
mango	0.95	1.00	0.98	21
mothbeans	0.91	0.81	0.86	26
mungbean	0.92	0.92	0.92	13
muskmelon	1.00	1.00	1.00	14
orange	1.00	1.00	1.00	25
papaya	1.00	0.95	0.98	21
pigeonpeas	1.00	0.78	0.88	18
pomegranate	1.00	1.00	1.00	20
rice	0.89	0.96	0.92	25
watermelon	1.00	1.00	1.00	21
accuracy			0.94	413
macro avg	0.95	0.95	0.95	413
weighted avg	0.95	0.94	0.94	413

Predict the Best Crop Based on Given Parameters

The saved model is utilized to predict the most suitable crop based on the environmental and soil conditions provided by the user. Input

features are collected from the user interface and passed to the trained machine learning model using ***model.predict()***.

The model processes these parameters and generates a crop recommendation that is best suited for the given conditions. This prediction helps farmers and agricultural users make data-driven decisions for improving crop productivity and efficient resource utilization.

```
prediction=model.predict((np.array([[105,35,40,25,64,7,160]])))

print("The suggested crop for given climatic condition is: ",prediction)
```

```
The suggested crop for given climatic condition is:  ['coffee']
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:439: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
  warnings.warn(
```

Epic 5: Application Building

This phase focuses on developing the web application and integrating itwith the trained machine learning model. The system accepts user inputs, generates crop predictions, and displays recommendations through an interactive interface.

Building HTML Pages

This section shows the navigation structure of the OptiCrop web application homepage. The HTML page contains a top navigation menu with links to Home, About, and FindYourCrop pages. The homepage introduces the purpose of the Smart Agricultural Production Optimization Engine and provides smooth navigation between different application modules.

```
<body>

<div class="topnav">
  <a class="active" href="#home">Home</a>
  <a href="{{ url_for('about') }}">About</a>
  <a href="{{ url_for('findyourcrop') }}">FindYourCrop</a>
</div>

<div style="padding-left:16px">
  <h2>Opti Crop Agricultural Optimisation</h2>
  <p>The main purpose of the OptiCrop aims to develop an advanced software system
</div>

</body>
</html>
```

This page presents detailed information about the OptiCrop project and explains the objectives of intelligent crop prediction using environmental and soil parameters. The About page describes how machine learning techniques are used to improve agricultural productivity, optimize farming decisions, and support sustainable agricultural practices.

```
<body>
<div style="padding-left:16px">
  <h2>Opti Crop Smart Agricultural Optimization Using ML</h2>
  <p>
    The project "Opti Crop: Smart Agricultural Production Optimization Engine" aims to develop an advanced
    software system that utilizes data-driven insights to optimize agricultural production for different crops.
    By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil
    temperature, humidity, pH, rainfall, and crop types, Opti Crop seeks to provide intelligent recommendations
    to farmers for maximizing yields and resource efficiency. The knowledge and insights gained through the
    Opti Crop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance
    the understanding of crop-environment interactions and inform the development of evidence-based agricultural
    strategies. The Opti Crop project revolutionizes agricultural practices by providing farmers with a smart
    production optimization engine. By integrating key environmental factors and crop-specific recommendations,
    Opti Crop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustaina
    agricultural practices.
  </p>
</div>
</body>
</html>
```

This section displays the crop prediction form where users provide agricultural parameters such as Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, and rainfall values. The HTML form

collects user input and sends the data to the backend prediction system for intelligent crop recommendation generation.

```
<h6>
  <center>
    <form action="{{ url_for('predict')}}"method="post">
        <label for="nitrogen" >nitrogen:</label><br>
        <input type="text" id="nitrogen" name="nitrogen" placeholder="nitrogen" required="required"/><br><br>
        <label for="phosphorous">phosphorous:</label><br>
        <input type="text" id="phosphorous" name="phosphorous" placeholder="phosphorous" required="required"/><b
        <label for="potassium">potassium:</label><br>
        <input type="text" id="potassium" name="potassium" placeholder="potassium" required="required"/><br><br>
        <label for="temperature">temperature:</label><br>
        <input type="text" id="temperature" name="temperature" placeholder="temperature" required="required"/><b
        <label for="humidity">humidity:</label><br>
        <input type="text" id="humidity" name="humidity" placeholder="humidity" required="required"/><br><br>
        <label for="ph">ph:</label><br>
        <input type="text" id="ph" name="ph" placeholder="ph" required="required"/><br><br>
        <label for="rainfall">rainfall:</label><br>
        <input type="text" id="rainfall" name="rainfall" placeholder="rainfall" required="required"/><br><br>

        <button type="submit" class="btn btn-primary btn-block ">Predict</button>
    </form>
    <br>
  </center>
</h6>
<h3>
      <p class="flask-output">{{ prediction_text }}</p>
</h3>
```

Build Python Backend Code

This section shows the backend initialization process using Flask and Pickle libraries. The trained machine learning model is loaded from the saved .pkl file, and the Flask application object is created to manage web routing and prediction requests

```
import numpy as np
from flask import Flask, request, render_template
import pickle
```

This section defines URL routes for the Home, About, and FindYourCrop pages using Flask decorators. Each route renders its

corresponding HTML template, enabling smooth navigation across the OptiCrop web application interface.

```
app=Flask(__name__)

path='C:/Users/siva/Documents/Uday/Projects/Opti_Crop/'
model=pickle.load(open(path+"model.pkl",'rb'))
```

This section handles user input received from the crop prediction form through a POST request. Agricultural parameters entered by the user are collected, converted into numerical values, and passed to the trained machine learning model using the predict() function to generate crop recommendations.

```
@app.route('/')

def home():
    return render_template('home.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/findyourcrop')
def findyourcrop():
    return render_template('findyourcrop.html')
```

This section starts the Flask development server using function. It enables the OptiCrop web application to run locally and allows users to interact with the crop prediction system through a browser interface.

```
@app.route('/predict',methods=['POST'])
def predict():
    int_features= [float(x) for x in request.form.values()]
    features=[np.array(int_features)]
    prediction=model.predict(features)

    output = prediction[0]
    return render_template('findyourcrop.html',prediction_text='Best crop for given conditions is {}'.format(output))
```

Main Function to run the application:

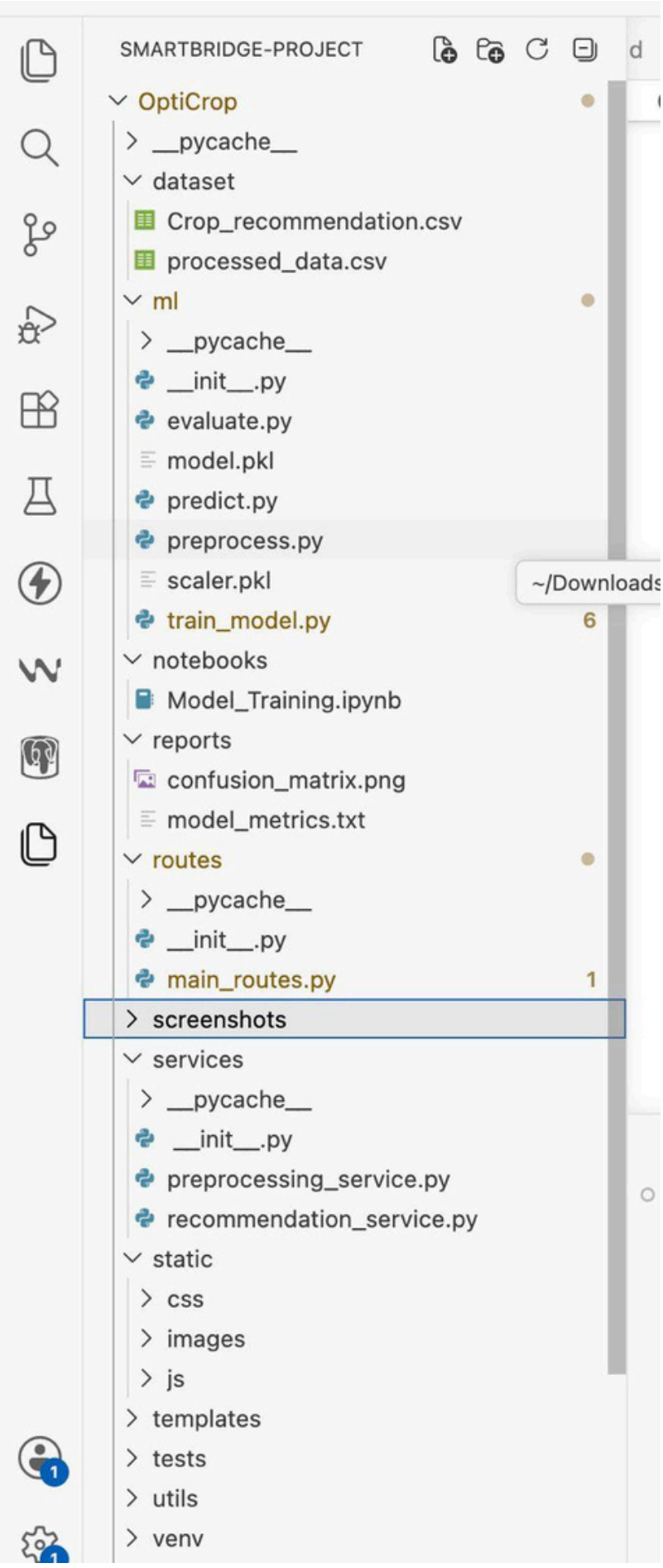
```
if __name__=="__main__":
    app.run(debug=True)
```

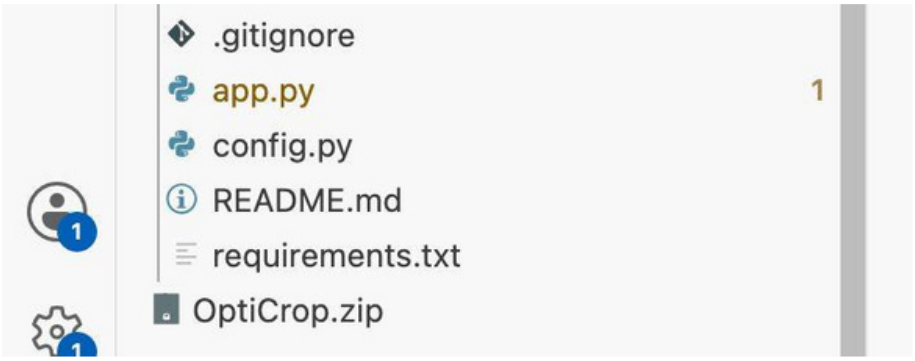
Run the Application

Open the project in Visual Studio Code and ensure all project folders are imported. Run ***app.py*** and click on the server URL shown in the terminal to open the application in your browser.

CODE FOLDERS :

CODE FOLDERS :





CODE :

```
15
    Windsurf: Refactor | Explain | ×
16  def load_artifacts():
17      """
18      Load the trained model and fitted scaler from disk using joblib.
19      Caches artifacts in memory after first load.
20
21      Returns:
22          tuple: (model, scaler)
23      """
24      global _model, _scaler
25
26      if _model is None:
27          _model = joblib.load(Config.MODEL_PATH)
28
29      if _scaler is None:
30          _scaler = joblib.load(Config.SCALER_PATH)
31
32      return _model, _scaler
33
```

```python
def predict_crop(n, p, k, temperature, humidity, ph, rainfall):
    """
    Predict the most suitable crop based on the provided input parameters.

    Args:
        n (float): Nitrogen content.
        p (float): Phosphorous content.
        k (float): Potassium content.
        temperature (float): Temperature in Celsius.
        humidity (float): Relative humidity percentage.
        ph (float): Soil pH value.
        rainfall (float): Rainfall in mm.

    Returns:
        str: Predicted crop label.
    """
    model, scaler = load_artifacts()

    input_df = pd.DataFrame(
        [[n, p, k, temperature, humidity, ph, rainfall]],
        columns=Config.FEATURE_COLUMNS,
    )

    scaled_input = scaler.transform(input_df)
    prediction = model.predict(scaled_input)

    return str(prediction[0])
```

Windsurf: Refactor | Explain | ×

```python
def get_candidate_models():
    """
    Define the candidate models to train and compare.

    Returns:
        dict: Mapping of model name to an untrained model instance.
    """
    return {
        "Logistic Regression": LogisticRegression(
            max_iter=1000, random_state=Config.RANDOM_STATE
        ),
        "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
        "Decision Tree Classifier": DecisionTreeClassifier(
            random_state=Config.RANDOM_STATE
        ),
        "Random Forest Classifier": RandomForestClassifier(
            n_estimators=200, random_state=Config.RANDOM_STATE
        ),
    }
```

```
Windsurf: Refactor | Explain | ×
def save_model_artifacts(model, scaler):
    """
    Save the trained model and fitted scaler using joblib.

    Args:
        model: Trained best-performing model.
        scaler: Fitted StandardScaler instance.
    """
    Config.MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, Config.MODEL_PATH)
    joblib.dump(scaler, Config.SCALER_PATH)

def preprocess_pipeline():
    """
    Run the full preprocessing pipeline: load, clean, save, split, scale.

    Returns:
        tuple: (X_train_scaled, X_test_scaled, y_train, y_test, scaler)
    """
    df = load_dataset()
    df = clean_dataset(df)
    save_processed_dataset(df)

    X, y = split_features_target(df)
    X_train, X_test, y_train, y_test = split_train_test(X, y)
    X_train_scaled, X_test_scaled, scaler = scale_features(X_train, X_test)

    return X_train_scaled, X_test_scaled, y_train, y_test, scaler
```

Windsurf: Refactor | Explain | ×

```python
def clean_dataset(df):
    """
    Handle missing values and remove duplicate records.

    Args:
        df (pd.DataFrame): Raw dataset.

    Returns:
        pd.DataFrame: Cleaned dataset.
    """
    df = df.dropna()
    df = df.drop_duplicates()
    df = df.reset_index(drop=True)
    return df
```

Windsurf: Refactor | Explain | ×

```python
def save_processed_dataset(df):
    """
    Save the cleaned dataset to the processed data path.

    Args:
        df (pd.DataFrame): Cleaned dataset.
    """
    Config.PROCESSED_DATASET_PATH.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(Config.PROCESSED_DATASET_PATH, index=False)
```

Windsurf: Refactor | Explain | ×

```python
def split_features_target(df):
    """
    Split the dataset into features and target.

    Args:
        df (pd.DataFrame): Cleaned dataset.

    Returns:
        tuple: (X, y) feature DataFrame and target Series.
    """
    X = df[Config.FEATURE_COLUMNS]
    y = df[Config.TARGET_COLUMN]
    return X, y
```

EXECUTING THE CODE :

```
((venv) ) jayanthbv@Jayanths-MacBook-Air OptiCrop % python app.py
 * Serving Flask app 'app'
 * Debug mode:
WARNING: This  . Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 112-448-809
^[
```

Follow link (cmd + click)

```
((venv) ) jayanthbv@Jayanths-MacBook-Air OptiCrop % python -m ml.train_model
Model comparison results:
  Logistic Regression: 0.9727
  K-Nearest Neighbors: 0.9795
  Decision Tree Classifier: 0.9795
  Random Forest Classifier: 0.9955

Best Model Selected: Random Forest Classifier
Model saved to: /Users/jayanthbv/Downloads/Smartbridge-project/OptiCrop/ml/model.pkl
Scaler saved to: /Users/jayanthbv/Downloads/Smartbridge-project/OptiCrop/ml/scaler.pkl
```


```
((venv) ) jayanthbv@Jayanths-MacBook-Air OptiCrop % python -m ml.train_model
Model comparison results:
  Logistic Regression: 0.9727
  K-Nearest Neighbors: 0.9795
  Decision Tree Classifier: 0.9795
  Random Forest Classifier: 0.9955

Best Model Selected: Random Forest Classifier
Model saved to: /Users/jayanthbv/Downloads/Smartbridge-project/OptiCrop/ml/model.pkl
Scaler saved to: /Users/jayanthbv/Downloads/Smartbridge-project/OptiCrop/ml/scaler.pkl
```

WEBSITE OVERVIEW :

